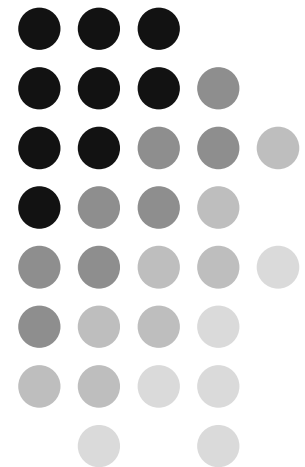


Relationnel - Objet



Motivations : Faiblesses du relationnel (1)

- Structure de données trop simple:
 - relation: tuple, 1FN, identifiant: valeur
 - Objets complexes, Non-1FN, identifiant d'objet
- Pas d'intégration des opérations avec les données :
 - pas de méthodes
 - pas d'encapsulation

Motivations : Faiblesses du relationnel (2)

- Pas de possibilité de référencer directement un objet composant (pointeur) par l'intermédiaire d'un oid
 - jointures sur identifiants de tables
- Pas de gestion des fonctions spécifiques à certaines données
 - spatial, multimédia,
 - très peu adaptable à ce type de données

Modèle Objet

- Objet (structure) complexe
- Identité d'objet
- Composition
- Hiérarchie de généralisation et héritage
- Collections
- Encapsulation

Solutions possibles

- Étendre le relationnel
 - Oracle
 - PostGrès
 - Informix
- Inventer de nouveaux SGBD: Orientés objets
 - O2,
 - Versant, ...

Modèle de données SQL3

- Objets possédant une valeur et un identifiant d'objet système (oid)
- Types de données utilisateur TAD avec des méthodes
- Attributs complexes et/ou multi-valués dans les tables (collections)
- Pouvoir établir des liens de composition entre tables via les oid
- Héritage

Exemple : table Personne

nom	{prénoms}	Date naissance	{voitures}			...
	prénom		modèle	année	no	
Duran	Pierre	16-05-1963	2ch	1975	128	
	Paul					
	Jacques		Mégane	1998	371	
Piona	Marie	29-02-1944	Twingo	1999	17	
	Jeanne					

Le modèle relationnel objet d'Oracle 9i

- Concepts NF² : structure complexe avec collection, Plus
- Possibilité de référencer les données (simili oid),
- Programmation incluse dans le SGBD avec les méthodes (langage : PL/SQL),
 - ⇒ Moins de jointure, références directes,
 - ⇒ Meilleures performances ! (en théorie)
- pas d'héritage dans v. 8i
- héritage dans v. 9i

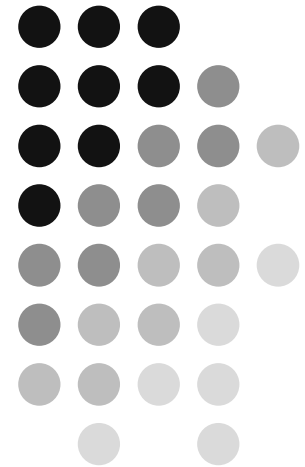
Concrètement

- 3 extensions par rapport au modèle relationnel :
 - Les TAD
(Types abstraits de données),
 - Types complexes
 - Composés,
 - Collections
 - Références,
 - Méthodes.
 - Les oids
 - L'héritage

PLAN

- I - Les TAD: différentes catégories de Types
- II - Création de tables à partir des types
- III - Définition d'attributs référence
- IV – Manipulation et interrogation

I – Les Types



I – Les TAD

- En relationnel: types prédéfinis
 - CHAR, VARCHAR, NUMBER, DATE, ...
- En relationnel – objet : nouveaux types définis par utilisateurs
 - ordre CREATE TYPE ...
- Plusieurs catégories de TYPES:
 - de type OBJET
 - de type COLLECTION
 - VARRAY
 - Nested Table

Types de "type OBJET"

- Objets complexes
- Informations incluses dans un type:
 - nom: identification de l'objet (unique dans le schéma)
 - attributs
 - méthodes: fonctions ou procédures écrites en PL/SQL ou Java et stockées dans la BD ou en C et stockées à l'extérieur de la BD

Create TYPE as OBJECT, syntaxe

- CREATE [OR REPLACE] TYPE type_name AS
OBJECT (att1 DATATYPE,
 att2 DATATYPE,
 ...
 attn DATATYPE ,
 [définition méthode])

/
- Remarque : après la création d'un TAD et uniquement
dans ce cas, '/' remplace le ';' pour terminer un ordre
SQL

Example

- CREATE TYPE Person AS OBJECT (
 name VARCHAR2(30),
 phone VARCHAR2(20));
- CREATE TYPE LineItem AS OBJECT (
 item_name VARCHAR2(30),
 quantity NUMBER,
 unit_price NUMBER(12,2));

Utilisation

```
CREATE TYPE Person AS OBJECT (  
    name VARCHAR2(30),  
    phone VARCHAR2(20) );
```

- Les types objet peuvent être utilisés comme
 - domaine de valeur d'une colonne d'une table
 - Create table Contacts (contact Person, date DATE)
 - domaine de valeur d'un attribut d'un autre type défini par l'utilisateur
 - Create type Contacts (contact Person, date DATE)
- domaine de relation
 - create table LesPersonnes of Person
 - chaque ligne de la table LesPersonnes: type Person

Types de "type COLLECTION"

- Type Tableau et type TABLE
 - VARRAY et NESTED TABLE
- Permettent de créer des attributs multi-valués
- Ils ont des constructeurs de même nom que le nom du type
 - NOM_TYPE (val1, val2, ..., valn)

VARRAYs

- Tableau à une dimension i.e. ensemble ordonné d'éléments de même type
- Tableau de taille variable (variable-array)
- Syntaxe:
 - CREATE [OR REPLACE] TYPE *nom_type*
AS VARRAY (*nb_max*) of *nom_type2*
- Définit un type tableau nommé *nom_type* de maximum *nb_max* éléments de type *nom_type2*
- *nom_type2* peut être n'importe quel type oracle ou créé par l'utilisateur

Exemple

- CREATE TYPE Tprices
AS VARRAY(10) OF NUMBER(12,2);
- Le type VARRAY nommé TPRICES ne peut pas posséder plus de 10 éléments,
- Chaque élément est de type NUMBER(12,2).
- Create table Product (nb Number,
price TPrice)

Utilisation

```
CREATE TYPE Telephones  
AS VARRAY(3) OF NUMBER(10);
```

- Les types VARRAYs peuvent être utilisés comme
 - domaine de valeur d'une colonne d'une table
 - Create table Personne (nom VARCHAR(20),
telephone TELEPHONES)
 - domaine de valeur d'un attribut d'un autre type défini par l'utilisateur
 - Create type Personne (nom VARCHAR(20),
telephone TELEPHONES)
- Aussi dans une autre collection (collection de collection)

Exemple plus complexe

```
CREATE TYPE cust_address_typ AS OBJECT (  
    street_address VARCHAR2(40),  
    postal_code VARCHAR2(10),  
    city VARCHAR2(30),  
    state_province VARCHAR2(10),  
    country_id CHAR(2)
```

```
CREATE TYPE address_array_t AS VARRAY(3) OF  
    cust_address_typ;
```

```
CREATE TABLE Customers (id_cust NUMBER, addresses  
    address_array_t);
```

Exemple

- Plusieurs prénoms pour une personne :
 - Create type liste-prenom as Varray(10) of varchar2(15)
/
Create type Tpersonne as object
 (nom varchar2(15), Date_n Date,
 prenoms liste_prenom,
 voiture Tvoiture)
/
Create table personnes of Tpersonne;

NESTED TABLE

- Ensemble non ordonné d'éléments de même type
- Syntaxe:
 - CREATE [OR REPLACE] TYPE nom_type AS TABLE OF nom_type2
- Définit un type table nommé *nom_type* dont chaque élément est une table (relationnelle) dont chaque tuple est de type *nom_type2*
- *nom_type2* peut être n'importe quel type oracle ou créé par l'utilisateur

Example (1)

- CREATE TYPE LineItem AS OBJECT (
 item_name VARCHAR2(30),
 quantity NUMBER,
 unit_price NUMBER(12,2));
- CREATE TYPE LineItem_table AS TABLE
 OF LineItem;
 - LineItem_table est une nested table

Utilisation

```
CREATE TYPE TPerson AS OBJECT (  
    name VARCHAR2(30),  
    firstname VARCHAR2(20) );  
CREATE TYPE TEnfants as TABLE of TPerson
```

- Les types NESTED TABLES peuvent être utilisés comme
 - domaine de valeur d'une colonne d'une table
 - Create table Personnes (nom VARCHAR(20),
enfants TEnfants)
NESTED TABLE enfants STORE AS Enfant_tab;
- Oracle crée physiquement une table annexe pour la "nested Table" (que l'utilisateur doit nommer)
 - ici, Enfant_tab

Utilisation

```
CREATE TYPE TPerson AS OBJECT (  
    name VARCHAR2(30),  
    firstname VARCHAR2(20) );  
CREATE TYPE TEnfants as TABLE of TPerson
```

- Les types NESTED TABLES peuvent être utilisés comme
 - domaine de valeur d'un attribut d'un autre type défini par l'utilisateur
 - Create type Personne (nom VARCHAR(20),
enfants TEnfants)
 - Aussi dans une autre collection (collection de collection)

Remarque

- Construction d'une Nested Table :
 - Génération automatique d'un identifiant `Nested_Table_Id` pour chaque collection
- Possibilité de créer un index dans la Nested Table
- Attention : Une Nested Table n'est pas une table objet
 - Pas d'oid associé à un élément de collection

Example (2)

```

Create type tvoiture as object
    (modele varchar2(15), annee date, no integer)
/
Create type ens_voiture as Table of Tvoiture
/
Create table Personnes (nom varchar2(15),
    prenom liste_prenom, date_naissance Date,
    voitures ens_voiture)
    Nested table voitures Store As ToutesVoitures;
  
```

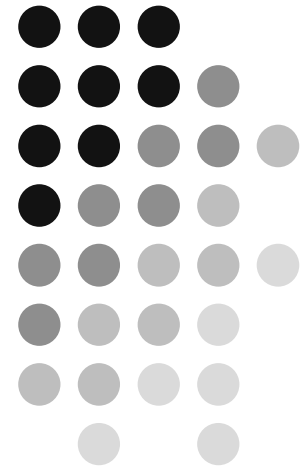
Personnes

Nom	liste-prénoms	age	voitures
'Liera'	('Pierre','Paul')	32	Id1
'Liera'	('Zoé','Marie')	27	Id2

Nested_table_Id	Tvoiture			
Id1	'2ch'	'1975'	12	
Id1	'206'	'2000'	3	
Id2	'2ch'	'1975'	12	
Id1	'607'	'2002'	1	

index ↗

II - Tables



II – Création de TABLES

- En oracle 9i, on peut créer:
 - (1) des tables relationnelles "classiques"
 - (2) des tables relationnelles en non 1^{ère} forme normale (NF²)
 - (3) des tables d'objet

Tables relationnelles "classiques"

- Table Personne (nom Varchar2(20),
prénom varchar2(20),
...)
- Les types utilisés comme domaine des attributs sont des attributs prédéfinis d'oracle
- e.g. Number, Varchar, Varchar2,...
- pas d'oid

Tables relationnelles en NF2

- Create table nom_table (att₁ nom_type₁,
att₂ nom_type₂, ...)
 - avec nom_type_n est le nom d'un TAD (varray,
nested table ou objet)
- Exemple:
CREATE TYPE telephones AS VARRAY(3) OF NUMBER(10);
CREATE TABLE Personne (nom VARCHAR(20),
téléphone telephones)
- pas d'oid

Tables d'objet (1)

- CREATE TYPE Person AS OBJECT (
 name VARCHAR2(30),
 phone VARCHAR2(20))/
- CREATE TABLE Person_table OF Person;

Tables d'objet

```
CREATE TYPE Person
      AS OBJECT (name VARCHAR2(30),
                  phone VARCHAR2(20) )
/
CREATE TABLE Person_table OF Person;
```

- Cette table peut être vue comme
 - Table possédant une seule colonne de type objet Person, sur laquelle vous allez pouvoir faire des opérations OO
 - select VALUE (p) from Person_table p
where p.name = "Smith";
 - Table possédant 2 colonnes, name et phone qui pourront être accédées de façon classique comme en relationnel
 - insert into Person_table values ("Smith", "079...");

Tables d'objet (3)

- Une **table d'objet** est:
 - Uniquement créable avec la commande :
create table nom_table of nom_TAD;
 - Toute instance d'une telle table possède un **oid** unique, ce sont des n-uplets objet.
 - La portée de cet oid est globale.
- Attention:
 - Les autres tables (rel. "classique", 1) et (rel. NF², 2) ne sont pas des tables d'objets.
 - Leurs instances n'ont pas d'oid

Résumé

- (1) Table relationnelle "classique"
 - CREATE TABLE Personne (
 nom VARCHAR(20),
 telephone number(10))
- (2) Tables relationnelles NF²
 - CREATE TABLE Personne (
 nom VARCHAR(20),
 telephone Ttelephones)
- (3) Tables d'objet
 - CREATE TABLE Person_table OF TPerson;

Valeurs par défaut

- Il est possible de définir des valeurs par défaut pour un TAD dans sa définition :

Create table personnes

(nom varchar2(15),

prenom varchar2(15),

date_naissance date,

**voiture Tvoiture DEFAULT voiture ('2ch',
'1975', 12))**

Contraintes

- Possible d'associer des contraintes aux tables objet comme aux tables "classiques"

```
CREATE TYPE person (  
    ssno NUMBER,  
    name VARCHAR2(100),  
    address VARCHAR2(100),  
    office TLocation );
```

```
CREATE TYPE TLocation (  
    building_no NUMBER,  
    city VARCHAR2(40) );
```

```
CREATE TABLE person_extent OF person (  
    ssno PRIMARY KEY );
```

Contraintes (suite)

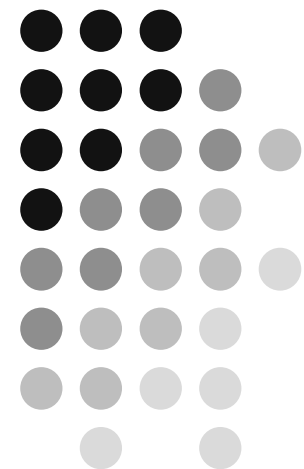
```
CREATE TYPE TLocation (  
    building_no NUMBER,  
    city VARCHAR2(40) );
```

- CREATE TABLE department (
 deptno CHAR(5) PRIMARY KEY,
 dept_name CHAR(20),
 dept_mgr person,
 dept_loc TLocation,
 CONSTRAINT dept_loc_cons1
 UNIQUE (dept_loc.building_no, dept_loc.city),
 CONSTRAINT dept_loc_cons2
 CHECK (dept_loc.city IS NOT NULL));

TAD interdépendants

- Si types inter-dépendants, impossible de créer un TAD complet et correct sans que l'autre ne soit déjà créé => problème.
- Exemple :
 - TAD Tpersonne utilise Tvoiture, et TAD Tvoiture utilise Tpersonne.
- Solution de Oracle 9i : déf. de TAD incomplet:
Syntaxe : create type nom_type/
 - -- ni attribut, ni méthode déclarés.
- TAD complété plus tard, mais est déjà référençable dans d'autres TAD.

III - Les Références



III – Les références

- Chaque instance d'une table d'objets possède un **identificateur d'objet** qui :
 - Identifie l'objet stocké dans ce n-uplet de façon unique,
 - C'est une colonne cachée, générée par le système,
 - Sert à référencer l'objet,
 - Oracle lui associe un index.

Référence

- Référence: Pointeur vers une instance d'une **table objet**
 - OID utilisé pour faire des références vers des objets
 - Attention, impossible de référencer une collection !
- Création d'une référence:
 - Créer le TAD t dont on veut référencer les instances,
 - Créer la table contenant ces instances de t,
 - Puis :
 - Créer le TAD qui référence t (mot-clé REF),
 - Créer la table associée.
 - ou
 - Créer directement la table.

Example

- Référencer la voiture d'une personne :

```
Create type Tvoiture as object (  
    modele varchar2(15),  
    annee date,  
    No integer)
```

```
/
```

```
Create table voitures of Tvoiture;
```

```
Create type Tpersonne as object  
    (nom varchar2(15),  
    prenom liste_prenom,  
    date_naissance Date,  
    voiture REF Tvoiture)
```

```
/
```

```
Create table personnes of Tpersonne;
```

Exemple – suite :

Création directe de la table

- Référencer la voiture d'une personne :

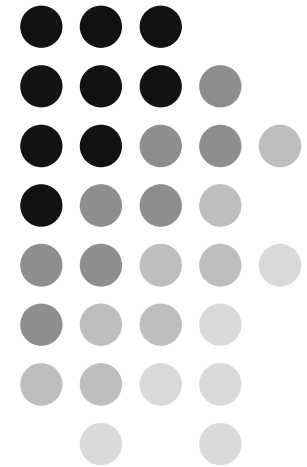
```
Create type Tvoiture as object  
(modele varchar2(15),  
annee date, No integer)  
/
```

```
Create table voitures of Tvoiture;  
-- voitures est une table d'objets.
```

```
Create table personnes  
(nom varchar2(15), prenom liste_prenom,  
date_naissance Date, voiture REF Tvoiture);  
-- personnes n'est pas directement construite  
avec un TAD, donc n'est pas une table d'objets.
```

IV - Manipulation et interrogation de TAD

- 1 - Références
- 2 - Tables d'objet
- 3 - Collections



Manipulation des références

- Deux opérateurs
 - REF : permet d'obtenir la référence associée à un objet (son oid)
 - REF : tuple -> référence vers un objet
 - Deref: permet de passer de la référence aux valeurs de l'objet
 - Deref : référence vers un objet -> valeur de l'objet

Opérateur REF

- Pour insérer un n-uplet dans la table personnes, il faut référencer la voiture de cette personne dans la table voitures
 - 1. Récupérer l'adresse de cette voiture pour la placer dans la table Personne.
 - = Utilisation de l'opérateur REF qui renvoie l'adresse d'un n-uplet dans une table

```
Create type Personne as object
(nom varchar2(15),
 prenom liste_prenom,
 date_naissance Date,
 voiture REF Tvoiture)
```

Opérateur REF (suite)

```
Insert into Personnes values ('Duran',  
    liste_prenom('Pierre', 'Paul', 'Jacques'),  
    '16-05-1963',  
    (select REF(v)  
    from voitures v  
    where v.modele = '2ch'));
```

Create type Personne as object
(nom varchar2(15),
 prenom liste_prenom,
 date_naissance Date,
 voiture REF Tvoiture)

Tpersonne			
Nom	{prénoms}	Date_naissance	@voiture
	Liste_prénom		

Table voitures :

modèle	année	No
2ch	1975	128
Mégane	1998	371

Tvoiture		
modèle	année	No

REF

Table
Personnes :

Nom	{prénoms}	Date_naissance	@voiture
	Liste_prénom		
Duran	Pierre	16-05-1963	
	Paul		
	Jacques		
Piona	Marie	29-02-1944	
	Jeanne		

Opérateur Deref

- Résultat de la requête : select * from personnes;

Nom	{prenoms}	Date_naissance	@voiture	
	Liste_prenom			
Duran	Pierre	16-05-1963	03F43970135A39847C... -- adresse du pointeur. Ne veut rien dire !	
	Paul			
	Jacques			

Opérateur Deref (suite)

- Select p.nom, p.prenoms, **Deref**(p.voiture)
from personnes p
where p.voiture.année < 1990;

Nom	{prenoms}	@voiture	
	Liste_prenom		
Duran	Pierre	TVoiture(2ch, 1975,128)	
	Paul		
	Jacques		

Référencer du vide

- Références qui n'existent plus
- Pour tester l'existence d'une instance référencée :
 - Utilisation du prédicat IS DANGLING.
- Exemple :
Update Personnes set voiture = NULL
Where voiture IS DANGLING

Insertion de données dans une table d'objets (1)

- Create type Tvoiture as object
(modele varchar2(15),
annee date,
no integer)
- Create table voitures of Tvoiture;
-- voitures est une table d'objets.
- L'insertion des données peut se faire comme d'habitude en SQL :
 - Insert into voitures values ('2ch', '1975', 128);

Sélection de données (2)

- Create type Tvoiture as object
(modèle varchar2(15),
année date, no integer)
- Create table voitures of Tvoiture;
- SQL: select * from voitures

Modèle	Année	No
2ch	1975	128
Mégane	1998	371

Table d'objets avec TAD

- Create type Tvoiture as object
(modèle varchar2(15),
année date, no integer)
- Create type Tpersonne as object
(nom varchar2(15), prénom varchar2(15),
date_naissance date, voiture Tvoiture)/
- Create table personnes of Tpersonne;

Tpersonne					
Nom	prénom	Date_naissance	Voiture		
			modèle	année	No

TAD, insertion de données

- | Nom | prénom | Date_naissance | Voiture | | | |
|-------|--------|----------------|---------|-------|----|--|
| | | | modèle | année | No | |
| Duran | Pierre | 16-05-1963 | 2ch | 1975 | 12 | |
| Piona | Marie | 29-02-1944 | Twingo | 1999 | 17 | |

- Insertion des données dans une table avec TAD :

- Insert into personnes values (Tpersonne('Duran', 'Pierre', '16-05-1963', Tvoiture('2ch', '1975', 12)));

← facultatif

← obligatoire⁵⁷

Table NF²

- Création d'une table NF² utilisant un TAD :
 - Create table personnes
(nom varchar2(15), prénom varchar2(15),
date_naissance date, voiture Tvoiture)
- Insertion de données dans une table utilisant un TAD :
 - Insert into personnes values ('Duran', 'Pierre',
'16-05-1963', Tvoiture('2ch', '1975', 12));

Opérateur Value

```
Create type Tprojet as object (pno  
NUMBER, pnom VARCHAR2(20))  
/  
create table Projets of Tprojet;
```

- Avec select : récupération de l'ensemble de valeurs des attributs d'une table.
 - SQL> Select * from Projets;
1 | 'Amber'
2 | 'Murmur'
- Si on veut récupérer la valeur structurée de l'objet:
Utilisation de l'opérateur VALUE
 - SQL> Select value(p) from Projets p;
Tprojet(1, 'Amber')
Tprojet(2, 'Murmur')

Manipulation de collections

- Insertion et mise à jour d'une nouvelle valeur pour la collection entière
 - OK pour varray et nested table
- Insertion, mise à jour, suppression d'éléments d'une collection
 - OK pour nested table uniquement

Insertion dans un Varray

- Insert into personnes values (Tpersonne('Duran',
liste_prenom('Pierre', 'Paul', 'Jacques'),
 '16-05-1963', Tvoiture('2ch', '1975', 12)));

obligatoire

Nom	{prenoms}	Date_naissance	Voiture		
	Liste_prenom		modele	annee	No
Duran	Pierre	16-05-1963	2ch	1975	12
	Paul				
	Jacques				
Piona	Marie	29-02-1944	2ch	1975	12
	Jeanne				

Insertion d'une "nested table"

```
Create type liste-prenom as Varray(10) of varchar2(15)
Create type Tvoiture as object (modele varchar2(15), annee date)
Create type ens_voiture as Table of Tvoiture
Create table Personnes (nom varchar2(15),
                prenom liste_prenom, date_naissance Date,
                voitures ens_voiture);
```

- on utilise le nom du type, ici **ens_voiture** :

- Exemple:

```
insert into Personnes values('mila',
    liste_prenom('Karim', 'Amin'), '16-03-75',
    ens_voiture (Tvoiture('2ch', 1975),
        Tvoiture('Mégane', 1998)));
```

Mise à jour dans une "nested table"

```
Create type liste-prenom as Varray(10) of varchar2(15)
Create type Tvoiture as object (modele varchar2(15), annee date)
Create type ens_voiture as Table of Tvoiture
Create table Personnes (nom varchar2(15),
      prenom liste_prenom, date_naissance Date,
      voitures ens_voiture);
```

- Supprimer une nested table :
 - Update Personnes SET voitures = NULL where nom= 'mila'
- Créer un ensemble vide :
 - Update Personnes SET voitures = ens_voiture()
where nom= 'mila'

Mise à jour dans une "nested table"

- Créer un singleton :
 - Update Personnes
SET voitures = ens_voiture(Tvoiture('Mégane',
1998, 179))
where nom= 'mila'

Opérateur TABLE

- utilisation : voire une collection comme une table.
 - 1) permet d'insérer des éléments dans une nested table
 - 2) permet de parcourir les tables ainsi obtenues.
 - Parcours de chaque collection de chaque n-uplet.
Ici, parcours (sélection) de chaque collection 'prénoms' et 'voitures' de chaque personne de la table personnes.

Insertion d'une valeur dans une Nested Table

- Create type Tprojet as object (pno NUMBER, pnom VARCHAR2(20))
/
Create type **Liste_projets as Table** of TProjet
/
Create table Employes (eno NUMBER,
 projets liste_projets)
 nested table projets store as LesProjets;
- INSERT INTO TABLE(SELECT e.projets
 FROM Employes e
 WHERE e.eno = 100)
VALUES (1, 'Project Neptune');
Ajout du tuple (1, 'Project Neptune') dans la table projets de l'employé n°100

Update dans une nested table

- Create type Tprojet as object (pno NUMBER, pnom VARCHAR2(20))
/
Create type **Liste_projets as Table** of TProjet
/
Create table Employes (eno NUMBER,
 projets liste_projets)
 nested table projets store as LesProjets;
- UPDATE TABLE(SELECT e.projets
 FROM employes e
 WHERE e.eno = 100) p
 SET VALUE(p) = Tprojet(1, 'Project BD') WHERE p.pno = 1;
***Mise à jour du tuple (1, 'Project Neptune') dans la table projets:
nouvelle valeur (1, 'Project BD') de l'employé n°100***

Suppression dans une nested table

- Create type Tprojet as object (pno NUMBER, pnom VARCHAR2(20))
/
Create type **Liste_projets as Table** of TProjet
/
Create table Employes (eno NUMBER,
 projets liste_projets)
 nested table projets store as LesProjets;
- DELETE FROM TABLE(SELECT e.projects
 FROM employee e
 WHERE e.eno = 100) p

WHERE p.pno = 1;
Suppression du tuple de pno=1 dans la table projets de l'employé n°100

Requête sur collections

```
Create type liste_voiture as Varray(10) of Tvoiture /  
/  
Create type liste-prenom as Varray(10) of varchar2(15)  
/  
Create table personnes (nom varchar2(15),  
                        prenom liste_prenom,  
                        date_naissance Date,  
                        voitures liste_voiture);  
  
insert into personnes values('mila',  
                             liste_prenom('Karim', 'Amin'), '16-03-75',  
                             liste_voiture(Tvoiture('2ch', 1975, 128),  
                                              Tvoiture('Mégane', 1998, 179)));
```

- On veut exécuter des requêtes sur les collections de voitures ou de prénoms

Requêtes sur collections (suite)

```
Select p.nom, p.prenoms  
From personnes p;
```

NOM	PRENOMS
-----	---------

-----	-----
-------	-------

'Mila'	liste_prenoms('Karim', 'Amin')
--------	---------------------------------

'dupont'	liste_prenoms('Paul', 'Pierre')
----------	----------------------------------

Retourne une collection pour
chaque tuple

Requêtes sur collections (suite)

Select p.nom, v.*

From personnes p, TABLE (p.prenoms) v;

NOM	PRENOMS
-----	---------

'Mila'	'Karim'
--------	---------

'Mila'	'Amin'
--------	--------

'dupont'	'Paul'
----------	--------

'dupont'	'Pierre'
----------	----------

L'expression TABLE (alias.collection) peut utiliser tout alias déclaré à sa gauche dans la clause FROM

Requêtes sur les VARRAY

- Create type listeprenom as Varray(10) of varchar2(15)
- Create type Tpersonne as object (nom varchar2(15), prenom listeprenom)
- insert into personnes values ('dupont', listeprenom('pierre','paul','jean'));
- insert into personnes values ('durant', listeprenom('p','q','r'));
- Comment chercher une personne particulière?
 - select * from personnes p, TABLE(p.prenoms) pr where pr.column_value='paul';
 - select * from personnes p where 'paul' in (select * from TABLE(p.prenoms));

Exemples (1)

```
Create table personnes (nom varchar2(15),  
    prenom liste_prenom,  
    date_naissance Date,  
    voitures liste_voiture);
```

- Numéro des voitures des personnes (ne retourne que les personnes qui ont des voitures):

```
Select p.nom, v.No  
    from personnes p, Table (p.voitures) v;
```

- Numéro des voitures des personnes (retourne aussi les personnes qui n'ont pas de voiture):

```
Select p.nom, v.No  
    from personnes p, Table (p.voitures) (+) v;
```

Exemple (2)

```
Create table personnes (nom varchar2(15),  
    prenom liste_prenom,  
    date_naissance Date,  
    voitures liste_voiture);
```

- Numéro des voitures de Piona :

```
Select v.No  
    from personnes p, Table (p.voitures) v  
    where p.nom = 'Piona';
```

- Numéro des voitures de Piona (autre solution):

```
Select v.No  
    from Table (select p.voitures from  
        Personnes p where p.nom='piona') v;
```